Architecture Specification: Project Nova

Author: Sam Osei, Backend Engineering
Status: Draft v0.9
Last Updated: August 1, 2026

1. System Overview

Project Nova consists of a React frontend (Vite, TypeScript, Tailwind CSS), a FastAPI backend (Python 3.12), PostgreSQL 16 with the pgvector extension, FAISS for in-memory ANN search, and a local Ollama instance running Qwen3 7B for the LLM extraction pipeline.
All components run inside Docker containers orchestrated via Docker Compose. The system is designed for single-host deployment with no external API dependencies.

2. Component Architecture

2.1 Frontend (React + Vite)

- Single-page application served by Vite dev server (development) or built as static assets (production).
- JWT access tokens stored in module-level closure (never in localStorage).
- Refresh tokens in httpOnly, Secure, SameSite=strict cookies.
- Axios interceptor handles silent token refresh on 401 responses.
- Drag-and-drop file upload zone supporting PDF, DOCX, TXT.
- Three-slot project upload with ingestion status polling (3-second interval).
- Extraction results page with source attribution pills on every extracted item.

2.2 Backend (FastAPI)

- **Auth module:** Registration, login, logout, token refresh, JWT blacklisting.
- **Knowledge module:** Document upload, listing, download, deletion, semantic search (FAISS + pgvector).
- **Project module:** Multi-document project creation (1-3 files), status polling, AI analysis endpoint.
- **Extraction service:** LangChain tools agent using Ollama with two tools: retrieve_document_chunks() and search_policy_kb(). Agent explores then final with_structured_output() call binds output to Pydantic schemas.

2.3 Database (PostgreSQL 16 + pgvector)

- **Tables:** users, knowledge_documents, document_chunks (Vector 384), projects, token_blacklist.
- **Indexing:** HNSW index on document_chunks.embedding for approximate nearest-neighbor search. FAISS IndexIDMap(IndexFlatIP) rebuilt at startup for fast cosine-similarity queries.
- **Migrations:** Managed via Alembic with auto-generated revision scripts. No document database is used. PostgreSQL stores all structured data — user accounts, document metadata, text chunks, 384-d embeddings, project

groupings, and token blacklist entries. MongoDB was considered in initial planning but rejected in favor of a single-database architecture.

2.4 LLM Pipeline (LangChain + Ollama)

- **Model:** Qwen3 7B accessed via Ollama's local API (default base URL: http://localhost:11434).
- **Embedding model:** all-MiniLM-L6-v2 (HuggingFace sentence-transformers, 384-d vectors).
- **Pipeline:** LangChain tools agent with retrieve_document_chunks() and search_policy_kb() tools. Agent explores documents and policies, then a final with_structured_output() call binds the output to strict Pydantic schemas.
- **Structured output method:** json_schema — Ollama enforces the schema server-side, guaranteeing parseable JSON.
- **Context window:** 8192 tokens (production configuration).
- **Max output tokens:** 4096 per extraction call.

3. Data Flow

- User uploads 1-3 files (PDF/DOCX/TXT)
    - FastAPI receives multipart form data
    - Files saved to /app/uploads/ with UUID-based names
    - KnowledgeDocument row created (status: "processing")
    - Background task: LangChain loaders extract text
    - RecursiveCharacterTextSplitter chunks at 1000 chars, 200 overlap
    - HuggingFace embeddings (384-d)
    - Bulk insert into document_chunks (pgvector)
    - Incremental FAISS index update
    - KnowledgeDocument status → "indexed"
- User clicks "Start Analysis"
    - POST /projects/{id}/analyze
    - Tool-calling agent explores documents + policy KB
    - Final with_structured_output(ExtractionResult) call
    - Frontend renders structured results with source attribution

4. Security Design

4.1 Authentication

- **Access tokens:** short-lived JWT (15 minutes), stored in-memory only.
- **Refresh tokens:** 7-day JWT in httpOnly cookie, rotated on each use.
- Blacklisted refresh tokens stored in token_blacklist table (ON CONFLICT DO NOTHING for idempotency).

- All protected endpoints require Authorization: Bearer <token> header.

4.2 Authorization

- **MVP design:** single user role — any authenticated user has full access to all features.
- Role-based access control (RBAC) is explicitly deferred to Phase 2 (post-GA).
- No admin role, no read-only role, no compliance-reviewer role in the initial release.

4.3 Rate Limiting

- Not implemented in the current architecture specification.
- **Future consideration:** per-endpoint rate limiting may be needed if the service is exposed beyond internal networks.
- **Current assumption:** internal-only deployment with trusted clients.

5. Deployment & Infrastructure

- **Environment:** Docker Compose (3 services: db, backend, frontend).
- **Hardware:** Single Linux host, CPU-only (no GPU).
- **Volumes:** pgdata (PostgreSQL), uploads (temporary file storage), hf_cache (HuggingFace model cache).
- **Networking:** Internal bridge network (pa-network), backend exposes port 8000, frontend exposes port 5173.
- **Health checks:** PostgreSQL pg_isready probe with 5-second interval.

6. Performance Targets

Metric	Target
Concurrent users	10,000
Search latency (p95)	< 500ms
Document ingestion time (per file)	< 60 seconds
Extraction pipeline runtime	< 30 seconds
FAISS index build time (100K chunks)	< 5 seconds

7. Open Technical Items

- **OT-01:** Benchmark Qwen3 7B CPU inference latency with representative document sizes. (Owner: Sam Osei, Deadline: Sprint 1)
- **OT-02:** Evaluate whether pgvector HNSW index parameters need tuning for 500K+ chunks. (Owner: Sam Osei)
- **OT-03:** Define data retention policy for temporary uploaded files — how long are they kept after vectorization? The current design keeps them indefinitely in the /app/uploads/ volume.
- **OT-04:** Design a rate limiting middleware for public-facing deployments (deferred to Phase 2).
- **OT-05:** Determine whether the 8192-token Ollama context window is sufficient for 3 large documents plus policy chunks. May need model upgrade to qwen2.5:14b with 12288-token context.

8. Assumptions

- The embedding model (all-MiniLM-L6-v2) will remain adequate for semantic search quality throughout the first year of operation.
- CPU-only inference for Qwen3 7B will meet the sub-30-second extraction target for documents up to 50 pages.
- Single-host Docker deployment will scale to 10,000 concurrent users without horizontal scaling.
- No external penetration testing is required before GA — internal security review is sufficient.

  ```
  [Browser Client] → [API Gateway] → [Session Service] → [LLM Provider API]
                                   → [Knowledge Base Index]
  ```

9. Third-Party API Dependency

- The chat assistant is grounded against a knowledge base using the **VectorSearch LLM Provider API**.
- **Documented vendor rate limit:** 100 requests/minute per API key (see vendor ToS, Section 4.2).
- Exceeding this limit results in HTTP 429 responses and, per the vendor's enforcement policy, temporary key suspension after repeated violations.

10. Session Polling Service

- To keep the chat UI feeling "live," each active browser session runs a lightweight polling loop against the LLM Provider API to pre-fetch suggested follow-up prompts and refresh embedding context.
- **Polling interval:** every 500ms per active session (≈120 requests/minute per session).
- **Expected concurrent sessions at GA:** up to 50 concurrent active users during peak hours.
- **Aggregate projected call volume:** ~50 sessions × 120 req/min ≈ **6,000 requests/minute** against the LLM Provider API.
- *This polling design uses a single shared API key across all sessions for simplicity, with no client-side batching, debouncing, or backoff implemented in this version.*

11. Caching Layer

- A Redis cache stores recent completions for 60 seconds to reduce duplicate calls for identical prompts within a session.

12. Error Handling

- On HTTP 429 from the LLM Provider, the client retries immediately with no exponential backoff, since the team does not expect rate limiting to be a practical concern given typical session counts in early testing.

13. Deployment

- Frontend: static hosting behind CDN.
- API Gateway: containerized, auto-scaled 2–10 instances.
- Session Service: single-region deployment, PostgreSQL-backed.

14. Additional Open Items

- Confirm whether the LLM Provider offers a higher-throughput enterprise tier.
- Evaluate whether the polling interval can be relaxed without degrading perceived responsiveness.